

Machine learning

Probabilistic models and learning

Thomas D. Nielsen

Probabilistic modeling

Bayesian network syntax

Let $X_1, \dots, X_n$ be a collection of random variables. A Bayesian network over $X_1, \dots, X_n$ consists of

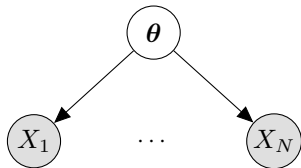
- a directed acyclic graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ whose nodes $\mathcal{V}$ are the variables $X_1, \dots, X_n$
- a set of local conditional distributions, $\mathcal{P} = \{p(X_i \mid \text{pa}(X_i)), X_i \in \mathcal{V}\}$, where $\text{pa}(X_i)$ are the parents of X_i in $\mathcal{G}$ as defined by the edges $\mathcal{E}$.

Bayesian network semantics

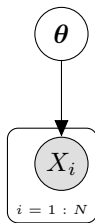
A Bayesian network $\mathcal{N}$ with nodes $X_1, \dots, X_n$ defines a joint distribution

$$p_{\mathcal{N}}(X_1, \dots, X_n) = \prod_{i=1}^n p(X_i \mid \text{pa}(X_i))$$

Two representations of a graphical model for N independent thumbtack tosses:



Unfolded notation



$$\theta \sim \text{Beta}(2, 5)$$

$$\theta \sim p(X_i = 1 | \theta) = \theta$$

Plate notation

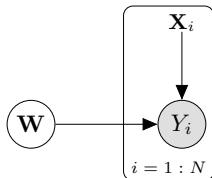
Syntactic sugar for representing repeated substructures

PGM part 0

Probabilistic modeling

- Specify prior beliefs
- Provide a probabilistic description/explanation of how the data is generated

Example



(Bayesian) linear regression model

- Only structure specified (using plate notation)

Bayes' rule Bayes' rule illustrates some of the inference problems we face:

$$\text{Posterior } p(\mathbf{W} | \mathbf{y}, \mathbf{x}) = \frac{\text{Likelihood } p(y | \mathbf{x}, \mathbf{W}) \text{ Prior } p(\mathbf{W})}{\int p(y, \mathbf{x}, \mathbf{W}) d\mathbf{W}} \\ \text{Marginal likelihood/model evidence}$$

Example: Bayesian linear regression

- $Y_i | \{\mathbf{w}, \mathbf{x}_i\} = \mathbf{w}^\top \mathbf{x}_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2)$ with $\mathbf{x}_i = [1, x_i]^\top$
- $\mathbf{w} \sim \mathcal{N}(\boldsymbol{\mu}_0 = \mathbf{0}, \boldsymbol{\Sigma}_0 = \mathbf{I}_{2 \times 2})$

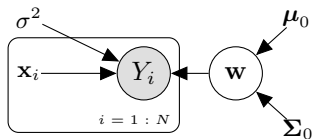
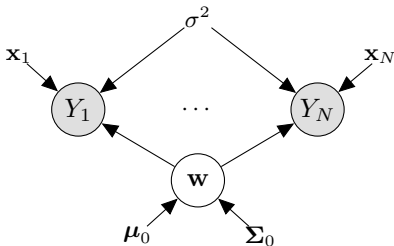
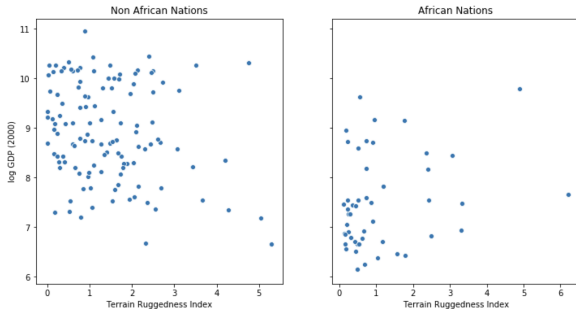


Plate notation



Unfolded notation

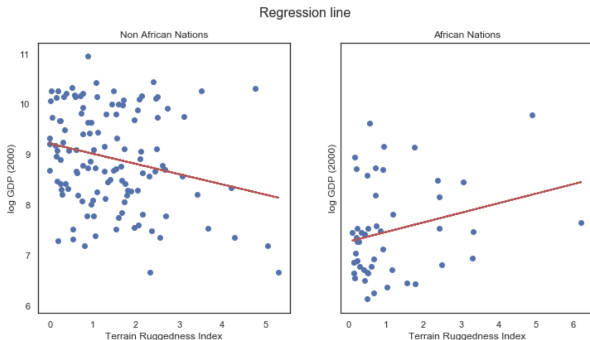
Real Data Example



Relationship between topographic heterogeneity and GDP per capita

- Terrain ruggedness or bad geography is related to poorer economic performance outside of Africa.
- Rugged terrains have had a reverse effect on income for African nations.

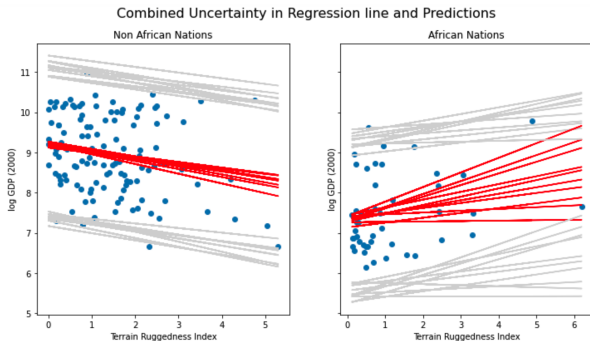
Real Data Example



Linear Regression Model

- Negative slope for Non African Nations.
- Positive slope for African Nations.

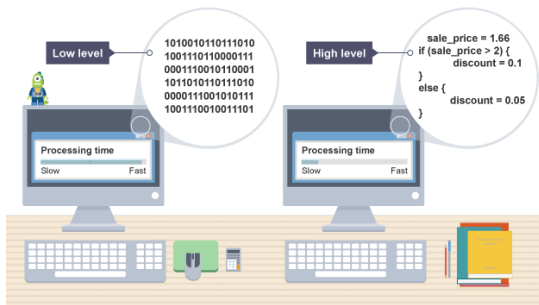
Real Data Example



Bayesian Linear Regression Model

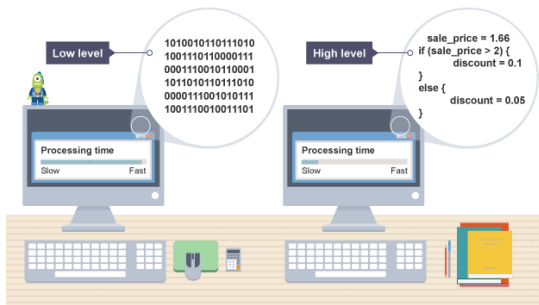
- Modeling data noise (aleatoric uncertainty).
- Modeling uncertainty about the linear coefficients (epistemic uncertainty).

Probabilistic programming



Similar situation 50 years ago:

- People used to program in low-level programming languages.
- Programming was complex and demand high-expertise.
- Focus more on low-level hardware details than application.



High-level programming languages brought many advantages:

- Programmers focused on the applications.
- Hardware Experts focused on compilers.
- High gains in productivity.
- “Democratization” of the software development.



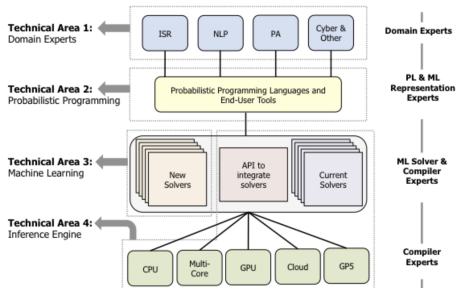
Claire D. Costa. Best Python Libraries for Machine Learning and Deep Learning.

<https://towardsdatascience.com/best-python-libraries-for-machine-learning-and-deep-learning-b0bd40c7e8c>

Big Data and Machine Learning Libraries:

- **High-quality**, well-maintained and open-source libraries
- They try to provide **high-level abstractions**.
- Hiding under the hood **low level details**.
- Increase the **adoption** of these technologies.

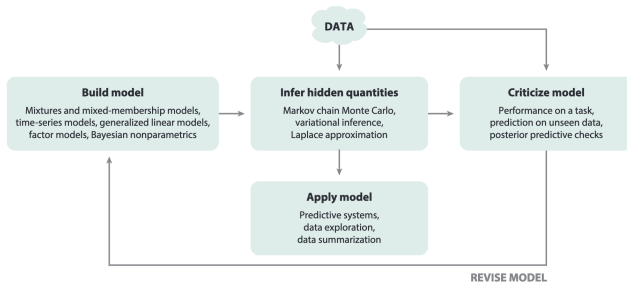
Why PPLs?



PPLs as high-level programming languages for **probabilistic machine learning systems**:

- Stacked architecture
- Different Domain Experts will code their models using the same language.
- ML experts will focus on the development of new ML solvers.
- Compile experts will focus on running these ML solvers on specialized hardware.

Box's Loop



Benefits of PPLs for developing probabilistic machine learning systems:

- Simplify probabilistic machine learning model code.
- Reduce development time and cost to encourage experimentation.
- Reduce the necessary level of expertise.
- “Democratization” of the development of probabilistic ML systems.

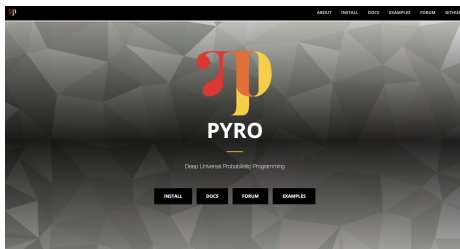


PyMC3



3rd Generation of PPLs :

- TensorFlow Probability, Pyro, PyMC3, InferPy, etc.
- Black Box Variational Inference and Hamiltonian Monte-Carlo.
- They scale to large data samples/high-dimensional models.
- Turing-complete probabilistic programming languages.
- Rely on deep learning frameworks (TensorFlow, Pytorch, Theano, etc).
 - Specialized hardware like GPUs, TPUs, etc.
 - Automatic differentiation methods.



Pyro's main features (www.pyro.ai) :

- Developed by UBER (the car riding company).
- Focus on deep generative models.
- Rely on Pytorch (Deep Learning Framework).
- Enable GPU acceleration and distributed learning.